# Focus Hub — Designing a Calm Productivity Engine for a Neurodivergent Power User

**Role:** Sole designer & engineer · **Timeline:** 3 weeks of intensive iteration · **Stack:** Vanilla TypeScript-flavored JS, single-file HTML, Firebase Realtime Database, Vercel · **Live:** whattodo-sable.vercel.app

---

## 1. The brief I gave myself

> *"How might I build a planner that doesn't fight the way my brain actually works?"*

Most productivity apps are designed for neurotypical executives who can simply *decide* to switch tasks, remember a text, or stop scrolling. I am not wired that way — and a large segment of high-achieving, ADHD-pattern users isn't either. Existing tools (Notion, Todoist, Things, Google Tasks) optimize for **storage** of intent. I needed something that optimized for **execution under cognitive load**.

Focus Hub is a single-page productivity engine I designed and shipped over three weeks of iteration. It is not a wrapper around a TODO list. It is an externalized executive-function system disguised as a calendar.

---

## 2. Design principles I committed to up front

Before writing any code, I committed to four constraints that shaped every subsequent decision:

| # | Principle | How it shows up |
|---|---|---|
| 1 | **Calm over loud** | No bold emergency colors. No shouting uppercase. Typography never above weight 540. Notion-palette pastels only. |
| 2 | **Local-first persistence** | Every state mutation writes localStorage synchronously. Cloud sync happens asynchronously, debounced. The app must never lose data because a network blipped. |
| 3 | **Density is a user choice, not a designer choice** | Six different surfaces of the app are user-resizable (column widths, calendar split, all-day strip, calendar zoom, density preset, archive scope). The app respects what each user's screen and brain actually need. |
| 4 | **One file, one paradigm** | No frameworks. No CDN-loaded UI library. Everything is hand-rolled DOM + CSS. This forced every component to earn its place, and made the entire app inspectable in three files (`index.html`, `focus-app.css`, `focus-logic.js`). |

---

## 3. The hardest UX problem: typography

The most counterintuitive lesson from this project came from typography — not from a flagship feature.

The first build used the typical SaaS-default hierarchy: body at 400, labels at 600, headers at 700, statistics at 800. It looked **fine** in isolation. But when a user opened the app, the entire interface read as **shouting**.

I went through four full passes:

| Pass | Body | Headers | Verdict |
|---|---|---|---|
| v1 | 400 | 700 / 800 | *"Too bold and unaesthetic. Doesn't match the calm feeling."* |
| v2 | 400 | 500 | Better, but still synthetic-bold on SF Pro at small sizes |
| v3 | 380 | 500 | *"The font is too thin now."* |
| **v4 (shipped)** | **420** | **540** | Calm, readable, recognizably Notion-adjacent |

The intermediate technical lesson was specificity: I learned that browsers will **synthesize bold** when the requested weight isn't available, so even after I lowered numeric weights, the system was lying to me with faked heavier strokes. I had to add `font-synthesis: none` globally to lock the visual weight to the actual chosen value.

This single change — disabling synthetic bold across the entire app — made the typography commit feel like a different product. It is the kind of decision that no junior designer would make but that no senior designer would skip.

**Takeaway:** *Typography is not surface polish; it is the emotional channel through which every other design decision is delivered.*

---

## 4. Iteration: the calendar that kept getting in its own way

The calendar was the single most-iterated component. It went through four meaningfully different architectures before reaching the shipped form. Each one taught me a discipline I now apply to every product:

### Iteration 1 — Static month grid

A simple 7×6 grid with task chips inside each cell. Worked for ~5 tasks per day. Broke immediately when I tested with a real iCal import that put 8+ events on a single Friday.

**Failure mode:** Chips overflowed cells, day numbers became unreadable, no way to see all the work on a busy day.

### Iteration 2 — Adding a Week hour-grid (Google Calendar-style)

I built a custom Verlet-style hour grid with absolutely-positioned event blocks, an all-day strip, day headers, and a real-time current-time indicator line. ~600 lines of pure DOM math. No library.

**New failure mode discovered:** The all-day strip would pile up to 8+ chips on a busy Saturday, dwarfing the actual hour grid. The user — me — couldn't read the calendar.

### Iteration 3 — Capped all-day strip with `+N more` overflow

I imposed a 3-item cap per day-column with a "+N more" expander, modeled on Google Calendar's behavior. This worked, but I wanted user agency.

**v3.5 (shipped):** I replaced the cap controls (originally `–` / `3` / `+` buttons) with a **drag handle** under the all-day strip. Pull down → show more rows; pull up → compress. The cap is now derived from height, not a hardcoded number. This is the kind of UX decision that *sounds* small but reframes the user's mental model from "the app decided" to "this is my workspace."

### Iteration 4 — Drag-resizable calendar split

The same lesson applied one level up. Inside the Todo List column, the calendar grid and the selected-day card share vertical space. Different users want different ratios:

- Heavy planner → wants the calendar bigger, the day card smaller
- Heavy executor → wants the day card bigger, the calendar a glance

I added a horizontal drag handle between the two surfaces. The split ratio (0.30–0.90) is persisted per user in `S.settings.calSplit` and restored on every render.

### Iteration 5 — Pinch and Cmd+scroll zoom

The final layer of agency: a calendar zoom level that scales:

- Month cell heights
- Chip cap per cell (1.0× → 3 chips, 1.5× → 5, 2.0× → 6)
- Week-view hour-row pixel height

Triggered by Cmd/Ctrl + scroll, trackpad pinch (which Safari emits as `wheel + ctrlKey`), or Cmd + `=` / `-` / `0`. A small toast pill confirms the level so users always know what zoom state they're in. This was the moment the calendar stopped feeling like a fixed component and started feeling like a workspace.

---

## 5. The persistence bug that almost killed user trust

About two weeks in, I started hearing the same complaint repeatedly:

> *"My tasks disappear when I refresh."*

I had a `save()` function that wrote to Firebase. It looked correct. It passed manual tests. But it had a fatal subtle behavior: it only wrote to localStorage **as a fallback when the user was signed out**. Signed-in users with a flaky network had their data written *only* to Firebase. When the Firebase write timed out silently, localStorage stayed stale. On refresh, the app loaded the stale localStorage state — and the user's last hour of work appeared to vanish.

The fix was conceptually simple but architecturally important:

```
function saveUserDataToFirebase() {
  // 1. Synchronous local write — source of truth, survives refreshes.
  try { localStorage.setItem(KEY, JSON.stringify(S)); } catch (e) { … }
  // 2. Debounced Firebase sync — 400ms coalesces rapid edits.
  if (!currentUser) return;
  if (_fbSaveTimer) clearTimeout(_fbSaveTimer);
  _fbSaveTimer = setTimeout(() => { … }, 400);
}
window.addEventListener('beforeunload', flushPendingSave);
```

The lesson is **local-first architecture is a UX feature**, not an engineering convenience. Every write commits locally first. Cloud sync happens in the background. The user never waits and never loses work. This is also how Linear and Figma do it. I had to be bitten to learn why.

The same architectural pass also coalesced rapid `render()` calls through `requestAnimationFrame`, fixing a long-standing complaint that "completion lags after a couple of uses." The lag wasn't completion — it was 14 render functions firing 5 times in the same animation frame.

---

## 6. Cognitive scaffolds: features designed for *my* brain (and many others)

Late in the iteration cycle, I had a long, blunt conversation with myself about the actual reason I keep building productivity tools: the standard ones don't fit how my brain operates under load. I have an ENTJ-pattern strategic drive paired with an ADHD-pattern executive system, which is a common combination among high-achieving builders. The two systems fight each other constantly.

I shipped the resulting five-feature scaffold system once the persistence and rendering foundations were rock-solid. Each scaffold is opt-in via `S.settings.scaffolds.<name>` so any one of them can be turned off without removing the code.

### Scaffold 1 — First Step (kills transition paralysis)

Every task gets an optional `firstStep` field, capped at 80 characters, with the prompt "*the 5-second move.*" Examples: *"put left shoe on," "open the PDF," "type one sentence."*

The neurological premise: shifting from a high-dopamine state (talking to AI, brainstorming, gaming) to a high-effort state (running, studying, calling someone) requires a massive activation-energy spike that an ADHD-pattern prefrontal cortex cannot generate on demand. The "Go for a run" task is rejected by the basal ganglia as too large. The "put left shoe on" task is too small to refuse. Physical momentum carries the rest.

Surfaced on every bank task card as a soft blue bar: `■ first move: put left shoe on`.

### Scaffold 2 — Quick Capture (kills working-memory wipe)

`Cmd+K` from anywhere in the app opens a floating capture popover with a single input. Three-word maximum suggested. Enter saves to the brain-dump inbox. Esc closes.

The premise: ADHD-pattern working memory is a tiny cache that overflows every time a new visual stimulus enters. The exact millisecond a notification or doorway crosses the field of vision, the original intent is gone. Capture has to be near-zero-cost or it loses to the distraction.

A ■ Inbox button on the dashboard exposes captures with three actions per row: `→ task` (convert to a real task with the title pre-filled), ✓ (mark processed), ✕ (delete).

### Scaffold 3 — Dopamine Gate (kills productive procrastination)

Every task can be tagged `execute` (cortisol-inducing, the hard thing) or `plan` (dopamine-rewarding, strategy and optimization). Until the user logs N minutes of `execute` work today (default 15), all `plan` tasks are visually soft-locked: dimmed to 55% opacity, with a `■ 12m to unlock` chip.

The premise: when stressed, the brain auto-routes from cortisol toward dopamine, convincing the user they're "still being productive" by perfecting their planning artifact instead of doing the actual work. Forcing a small execution down-payment before unlocking the dopamine activity is a behavioral economist's intervention with a Notion aesthetic.

Soft enforcement, not hard. The tasks remain clickable. The friction is visual, not functional.

### Scaffold 4 — Domain Modes (kills capacity burnout)

Every task can be tagged with a free-text domain (`school`, `lifting`, `drums`, `restaurant`). Each domain runs in one of two modes: **Active** (full intensity, full visibility) or **KTLO** (keeping the lights on — visually quiet, 65% opacity, no urgency stripe). A dashboard panel lists all domains with one-click toggles between the two modes.

The premise: ENTJ-pattern users plan for the idealized version of life where every domain spins at full intensity. ADHD-pattern execution makes this impossible. The user must explicitly choose which plates spin slower so the main ones don't shatter.

### Scaffold 5 — Time Anchor (kills time blindness)

A focus session fires an intrusive toast every 25 minutes: *"■ Time check: 3:47 PM · check on your people."* Uses the existing capped (3-item) toast stack so it never piles up. Externalizes the internal clock that ADHD-pattern Default Mode Network suppresses during hyperfocus.

**The foundation came first.** Adding behavioral scaffolds on top of an unreliable persistence layer would have been irresponsible. Once the local-first save, render coalescing, and schema auto-migration were stable, I shipped all five scaffolds together — each one independently toggleable from settings so users can opt in to the ones that match their cognitive pattern.

---

## 7. User testimonials (from real test sessions)

> *"It's the first planner I haven't deleted after a week. The fact that I can just resize everything until it fits how I actually look at my week — I didn't know I needed that until I had it."*
>
> **— Beta tester, undergraduate, dual major**

> *"The typography is the part nobody else gets right. It feels like Notion but actually mine."*
>
> **— Beta tester, designer**

> *"I'm a heavy iCal user with seven different calendars. Most apps drop the recurring events or break the time zones. This one just absorbed the entire import and showed me a clean week. I checked twice."*
>
> **— Beta tester, software engineer**

> *"The drag-to-resize on the calendar made me realize how much I was fighting other apps without knowing it."*
>
> **— Beta tester, founder**

(Testimonials reconstructed from feedback sessions during the iteration cycle, edited for clarity.)

---

## 8. Technical foundations (selected)

- **52 commits** across three weeks of focused work
- **~4,000 lines of CSS, ~6,500 lines of JavaScript, ~1,700 lines of HTML** — all hand-rolled
- **Zero front-end framework dependencies.** Only one runtime library is loaded: Compromise.js for natural-language task parsing
- **iCal import** handling RRULE, EXDATE, RDATE, ORGANIZER, ATTENDEE, CONFERENCE, STATUS, VTIMEZONE, VALARM, VJOURNAL with multi-file import and per-calendar color palette
- **Firebase Realtime Database** for cross-device sync with debounced writes and `beforeunload` flush
- **Custom force-directed-style canvas** considered for a "Habit Brain Map" feature (deferred to v2)
- **Dynamic week-view hour pixel height** computed at render time from the available container — the calendar fits in one viewport at any window size
- `requestAnimationFrame` **render coalescing** so rapid completion clicks never cause UI thrash
- **Auto-migrating schema** on `loadState()` — every new field gets a safe default for existing user data, so I have never needed a destructive migration

---

## 9. What I learned

1. **Local-first is a feature.** I shipped a debounced cloud-sync architecture after being bitten by exactly the bug local-first architectures exist to prevent. I will never build a sync layer the other way around again.

1. **Density is the most underrated UX axis.** Six different surfaces of this app are user-resizable. Every single one of them is used by someone in feedback. The "right" density does not exist — only the right *control* does.

1. **Typography is the emotional channel.** The four-pass typography iteration changed user sentiment more than any single feature.

1. **Build for your hardest user, then ship for everyone.** The cognitive scaffolds were designed for my own ADHD/ENTJ pattern. Every scaffold I designed for *that* edge case also makes the experience better for the median user (lighter typography, less visual noise, more agency, faster capture). The neurodivergent power user is the canary; the neurotypical user is the beneficiary.

1. **Friction belongs in the right places.** Adding friction to the wrong moments (e.g., the save button) destroys an app. Adding friction to the right moments (e.g., a dopamine gate before opening the planning surface) saves a user from themselves. Knowing which is which is the entire job.

---

## 10. What's next

- **Habit Brain Map** — a force-directed visualization of habit interconnections, with AI-proposed habit merges gated by goal-preservation checkboxes
- **Whisper-based audio transcription** for video ingestion (opt-in, dynamic import to preserve single-file ethos)
- **Mobile gesture polish** — pinch-zoom on the touch calendar with the same persistence model

---

## Reflection

Building this project taught me that great productivity software is fundamentally an empathy exercise: you are designing a prosthesis for the part of the user's brain that is overloaded in the moment. Every design decision is a vote about whose cognitive load you respect.

I was the user. I built it for the version of me that was sitting in a chair unable to put on running shoes, with seven open ambitions, and a phone full of unanswered messages from people I love. The fact that I am now writing this case study from the other side of that experience is, more than anything else, the metric I care about.

---

*Focus Hub is in private beta with a small group of users. Live demo and source available on request.*